

OOP Comparison: Simple Car Class vs Real-World EcommerceAnalyzer

One of the best ways to teach OOP is to show that **both classes follow exactly the same structure**, but one works with simple objects while the other works with business data.

1. Simple Car Class

Real Life Thinking

A car has:

- Properties (Brand, Model, Year)
- Actions (Start, Stop, Display Details)

class Car:

```
def __init__(
    self,
    brand,
    model,
    year
):
    self.brand = brand
    self.model = model
    self.year = year

def start(self):
    print(f"{self.brand} started")

def stop(self):
    print(f"{self.brand} stopped")

def display_info(self):
    print(
        f"Brand: {self.brand}, "
```

```
f"Model: {self.model}, "  
f"Year: {self.year}"  
)
```

Using the Class

```
car1 = Car(  
    "Toyota",  
    "Camry",  
    2024  
)
```

```
car1.start()  
car1.display_info()  
car1.stop()
```

Output:

```
Toyota started  
Brand: Toyota, Model: Camry, Year: 2024  
Toyota stopped
```

2. EcommerceAnalyzer Class

Real Life Thinking

An e-commerce company has:

- Orders
- Customers
- Categories
- Delivery Status

Actions:

- Create DataFrame
- Calculate Delivery Rate
- Find Frequent Returners
- Create Category Summary

```
class EcommerceAnalyzer:
```

```
def create_order_df(self, data):  
    pass  
  
def monthly_delivery_rate(self, df):  
    pass  
  
def add_return_flag(self, df):  
    pass  
  
def frequent_returners(self, df):  
    pass  
  
def category_order_summary(self, df):  
    pass
```

Side-by-Side Comparison

Simple OOP (Car)	Business OOP (EcommerceAnalyzer)
Represents one Car	Represents Order Analytics System
Stores Brand, Model, Year	Processes Order Data
start()	create_order_df()
stop()	add_return_flag()
display_info()	category_order_summary()
Works with simple values	Works with DataFrames
Returns strings	Returns analytics results

Understanding **self**

Car Example

```
class Car:

    def display_info(self):
        print(self.brand)
```

Python automatically passes:

```
car1.display_info()
```

Internally:

```
Car.display_info(car1)
```

Ecommerce Example

```
class EcommerceAnalyzer:
```

```
    def create_order_df(
        self,
        data
    ):
        return pd.DataFrame(data)
```

Call:

```
analyzer.create_order_df(data)
```

Internally:

```
EcommerceAnalyzer.create_order_df(
    analyzer,
    data
)
```

Constructor Comparison

Car Class Uses Constructor

```
class Car:
```

```
    def __init__(
        self,
```

```
    brand,  
    model  
):  
    self.brand = brand  
    self.model = model
```

Usage:

```
car = Car(  
    "Toyota",  
    "Camry"  
)
```

Object stores state:

```
brand = Toyota  
model = Camry
```

EcommerceAnalyzer Does NOT Need Constructor

```
class EcommerceAnalyzer:
```

```
    def create_order_df(self, data):  
        pass
```

Usage:

```
analyzer = EcommerceAnalyzer()
```

Why?

Because it doesn't need to remember:

```
CustomerID  
Category  
OrderStatus
```

Those values come from the DataFrame every time.

Car Example with State

```
class Car:
```

```
def __init__(
    self,
    brand
):
    self.brand = brand

def start(self):
    print(self.brand)
```

Object:

```
car = Car("Honda")
```

Memory:

```
car
└── brand = Honda
```

Ecommerce Example Without State

class EcommerceAnalyzer:

```
def category_order_summary(
    self,
    df
):
    return pd.crosstab(
        df["Category"],
        df["OrderStatus"]
    )
```

Object:

```
analyzer = EcommerceAnalyzer()
```

Memory:

```
analyzer
└── No stored business data
```

The DataFrame is passed into methods when needed.

Better Comparison: Student Example

This is closer to what trainees understand.

Traditional OOP

class Student:

```
def __init__(
    self,
    name,
    marks
):
    self.name = name
    self.marks = marks

def calculate_grade(self):

    if self.marks >= 90:
        return "A"

    return "B"
```

Usage:

```
student = Student(
    "Rahul",
    95
)

print(
    student.calculate_grade()
)
```

Output:

A

Data Engineering Equivalent

class StudentAnalyzer:

```
def calculate_grade(
```

```
self,
df
):

df["Grade"] = (
    df["Marks"]
    .apply(
        lambda x:
            "A" if x >= 90
            else "B"
    )
)

return df
```

Usage:

```
analyzer = StudentAnalyzer()
```

```
result = analyzer.calculate_grade(df)
```

Why Data Engineers Still Use Classes

Without OOP:

```
create_order_df()
```

```
monthly_delivery_rate()
```

```
frequent_returners()
```

```
category_order_summary()
```

Many independent functions.

With OOP:

```
analyzer = EcommerceAnalyzer()
```

```
analyzer.create_order_df()
```


analyzer.monthly_delivery_rate()

analyzer.frequent_returners()

analyzer.category_order_summary()

Everything is grouped under one business object.

Visual Diagram

SIMPLE OOP

Car

- |— brand
- |— model
- |— start()
- |— stop()
- |— display_info()

DATA ENGINEERING OOP

EcommerceAnalyzer

- |— create_order_df()
- |— add_return_flag()
- |— monthly_delivery_rate()
- |— frequent_returners()
- |— category_order_summary()

Key Message for Trainees

A class is simply a **container that groups related data and behavior together**.

Car Class

Data

- |— Brand
- |— Model

Behavior

|— Start
|— Stop

EcommerceAnalyzer Class

Data

|— Orders
|— Customers
|— Categories

Behavior

|— Create DataFrame
|— Calculate Delivery Rate
|— Find Returners
|— Generate Reports

The only difference is the **domain**:

- **Car** → manages a vehicle.
- **Student** → manages a student.
- **EcommerceAnalyzer** → manages e-commerce analytics.

The OOP concepts (**class**, **object**, **self**, methods, encapsulation) remain exactly the same.